

UNIT- 2 BASIC PYTHON SYNTAX

1. Python Reserved Words

- Python में कुछ words को Reserved Words या Keywords कहा जाता है।
- इनका use Python syntax में special meaning के लिए होता है।
- हम इन words को variable name, function name, class name या किसी identifier के रूप में use नहीं कर सकते।
- Python 3.x में कुल 35 Reserved Words होते हैं।

False	None	True	and	as	assert	async	await
break	class	continue	def	del	elif	else	except
finally	for	from	global	if	import	in	is
lambda	nonlocal	not	or	pass	raise	return	try
while	with	yield					

* Note: Reserved Words को हम बदल (rename) नहीं सकते, क्योंकि इनका उपयोग Python interpreter के द्वारा syntax और operations को define करने के लिए किया जाता है।

2. Naming Conventions (नामकरण के नियम)

- Python में variables, functions, classes, modules आदि के नाम निर्धारित करने के लिए कुछ नियम follow किए जाते हैं।
- सही naming code को readable, maintainable और professional बनाती है।
- Python में नामकरण के लिए निम्नलिखित प्रमुख नियम हैं:

① Meaningful Names (सार्थक नाम)

हमेशा ऐसा नाम रखें जो उसके काम को दर्शाए।

✓ total_marks, student_age
✗ tm, x1

② Use of Letters, Digits and Underscore

नाम में केवल letters (A-Z, a-z), digits (0-9) और underscore (_) का उपयोग करें।

✓ age10, student_1
✗ age@10, stu#dent

③ First Character Rule

नाम की शुरुआत letter (A-Z, a-z) या underscore (_) से होनी चाहिए, digit से नहीं।

✓ _value, total
✗ 1value, 2total

④ Case Sensitivity (Case Sensitive)

Python में upper case और lower case अलग-अलग माने जाते हैं।

total, Total और TOTAL
तीनों अलग variables हैं।

⑤ No Spaces Allowed

नाम में space नहीं होना चाहिए। इसके स्थान पर underscore (_) का उपयोग करें।

✓ total_marks
✗ total marks

⑥ Avoid Python Reserved Words

Reserved Words का उपयोग नाम के रूप में नहीं करें।

✗ class, def, return, if...
(यह Reserved Words हैं)

⑦ Recommended Style

- Variable और function names के लिए lower_case_with_underscores
- Class names के लिए CamelCase (First letter capital)
- Constant names के लिए UPPER_CASE का उपयोग करें।

✓ student_name
✓ CalculateSum()
✓ MAX_VALUE

* Summary: Reserved Words Python के special keywords होते हैं और इन्हें identifier के रूप में use नहीं किया जा सकता। सही Naming Conventions follow करने से हमारा code clean, readable, maintainable और professional बनता है।

1. String Values

- Python में String, characters (अक्षरों) का sequence होता है।
- String को single quotes (' '), double quotes (" ") या triple quotes (''' ''' या "" "" """) के द्वारा लिखा जा सकता है।
- String immutable होती है, अर्थात् एक बार create होने के बाद इसके characters को बदला नहीं जा सकता।

String Declaration के तरीके :

Example	Explanation
'Hello'	Single quoted string
"Hello"	Double quoted string
'''Hello'''	Triple quoted string (multi-line string के लिए उपयोगी)
"""Hello"""	Triple double quoted string

उदाहरण :

```
s1 = 'Python'
s2 = "Programming"
s3 = '''This is
multi-line
string'''
s4 = """This is also
multi-line
string"""
```

Indexing in String :

String के हर character का एक index होता है।

Index 0 से शुरू होता है।

s = 'Python'

P	y	t	h	o	n
index → 0	1	2	3	4	6

Negative Indexing : दायें से बायें की ओर -1 से शुरू होता है।

P	y	t	h	o	n
-7	-6	-5	-4	-3	-1

2. String Methods

- Python में कई built-in methods होती हैं जिनका उपयोग String पर अलग-अलग operations करने के लिए किया जाता है।
- नीचे कुछ महत्वपूर्ण String methods दिए गए हैं :

Method	Description	Example	Output
lower()	सभी characters को lower case में बदलता है।	'PYTHON'.lower()	'python'
upper()	सभी characters को upper case में बदलता है।	'python'.upper()	'PYTHON'
title()	हर word के पहले letter को capital बनाता है।	'hello world'.title()	'Hello World'
strip()	String के start और end से spaces हटाता है।	' Python '.strip()	'Python'
len()	String की length (कुल characters की संख्या) लौटाता है।	len('Python')	6
replace(old, new)	पुराने substring को नए substring से replace करता है।	'I love Python'.replace('love', 'like')	'I like Python'
find(sub)	Substring का first occurrence का index लौटाता है। न मिलने पर -1।	'Python Programming'.find('Pro')	7
split(sep)	String को separator के आधार पर list में तोड़ता है।	'a,b,c'.split(',')	['a', 'b', 'c']
join(iterable)	Iterable (जैसे list) के elements को String में जोड़ता है।	'-'.join(['a', 'b', 'c'])	'a-b-c'
startswith(sub)	क्या String दिए गए substring से start होती है? (True/False)	'Python'.startswith('Py')	True
endswith(sub)	क्या String दिए गए substring से end होती है? (True/False)	'Python'.endswith('on')	True
count(sub)	Substring कितनी बार आता है, यह बताता है।	'banana'.count('a')	3

उदाहरण :

```
s = ' Hello Python '
print(s.strip())      → 'Hello Python'
print(s.lower())      → ' hello python '
print(s.replace('Python', 'World')) → ' Hello World '
print(s.split())      → ['Hello', 'Python']
print('-'.join(['A', 'B', 'C']))  → 'A-B-C'
```

Note :

- * String immutable होती है, इसलिए कोई भी method original string को change नहीं करती, एक नई string return करती है।

1. The format() Method

- Python में `format()` method का उपयोग string formatting के लिए किया जाता है।
- यह method string में placeholder `{}` के स्थान पर values को insert करता है।
- Placeholder को index number या key name के द्वारा refer किया जा सकता है।
- यह method बहुत flexible और readable होता है।

Syntax :

"string with {} placeholders".format(value1, value2, ...)

या

"string with {key} placeholders".format(key=value)

Examples :

Code	Explanation	Output
① "My name is {}".format("Pankaj")	एक value insert की	My name is Pankaj
② "{} + {} = {}".format(2, 3, 5)	तीन values insert की	2 + 3 = 5
③ "{0} is {1} years old".format("Ravi", 20)	index number का उपयोग	Ravi is 20 years old
④ "{name} lives in {city}".format(name="Ram", city="Kanpur")	key name का उपयोग	Ram lives in Kanpur
⑤ "{x:.2f}".format(12.34567) # float formatting	float value को 2 decimal में	12.35
⑥ "{:<10} {:>10} {:^10}".format("left", "right", "center")	alignment (left, right, center)	left right center ----- ----- -----

* Note :

- * f-string (formatted string literal) Python 3.6+ में और भी आसान तरीका है।

ex. name = "Pankaj" age = 20

f"My name is {name} and I am {age} years old."

→ My name is Pankaj and I am 20 years old.

Advantage of format()

- Readable
- Reusable
- Supports both index & key
- Alignment & formatting options

2. String Operators

- Python में strings पर कई operators काम करते हैं।
- इनकी मदद से strings को combine, repeat, compare आदि किया जा सकता है।

Operator	Name	Description	Example	Output
+	Concatenation	दो strings को जोड़ता है	"Hello" + "World"	'Hello World'
*	Repetition	String को n बार repeat करता है	"Hi" * 3	'HiHiHi'
in	Membership	एक string दूसरी string में है या नहीं	'ell' in "Hello"	True
not in	Non-membership	एक string दूसरी string में नहीं है	'xyz' not in "Hello"	True
==	Equality	दो strings बराबर हैं या नहीं	"abc" == "abc"	True
!=	Inequality	दो strings बराबर नहीं हैं	"abc" != "def"	True
<	Less than	पहली string दूसरी से छोटी है (lexicographically)	"apple" < "banana"	True
>	Greater than	पहली string दूसरी से बड़ी है	"zoo" > "apple"	True
<=	Less than or equal	छोटी या बराबर	"cat" <= "cat"	True
>=	Greater than or equal	बड़ी या बराबर	"dog" >= "dog"	True

Examples :

```
s1 = "Python"
s2 = "Programming"
print(s1 + " " + s2) → 'Python Programming'
print(s1 * 2) → 'PythonPython'
print('thon' in s1) → True
print('java' not in s1) → True
print('A' < 'B') → True
```

Note :

- * String comparison lexicographical order में होता है, यानी dictionary order के अनुसार।
- * in और not in operators search के लिए बहुत useful होते हैं।
- * + और * सबसे अक्सर उपयोग होने वाले operators हैं।

1. Numeric Data Types

- Python में numbers को store करने के लिए अलग-अलग Numeric Data Types होते हैं।
- ये types immutable होते हैं, अर्थात इनका value change नहीं किया जा सकता।
- Python 3.x में मुख्यतः तीन Numeric Types होते हैं: int, float और complex।

Data Type	Description	Example	Type() Output	Size (Approx.)
int	Integer numbers (पूरी संख्या) Positive, Negative या Zero	10, -25, 0, 999	<class 'int'>	Variable (Depends on value)
float	Floating point numbers (दशमलव संख्या)	10.5, -3.14, 0.0, 1.2e3	<class 'float'>	24 bytes (Approx.)
complex	Complex numbers (a + bj के रूप में)	3+4j, -2.5j, 10+0j	<class 'complex'>	32 bytes (Approx.)

Notes :

- * int की size value के अनुसार बढ़ती या घटती है।
- * float IEEE 754 standard के अनुसार store होता है।
- * complex number में real part (a) और imaginary part (b) होता है, $j = \sqrt{-1}$ ।

Examples :

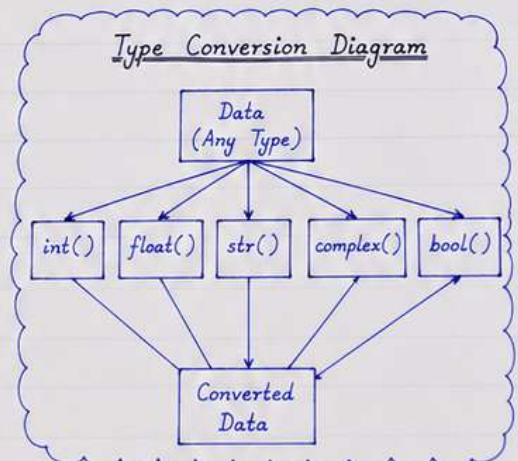
```
>>> type(10)
<class 'int'>
>>> type(3.14)
<class 'float'>
>>> type(2+5j)
<class 'complex'>
```

2. Conversion Functions

- Conversion functions का उपयोग एक data type को दूसरे data type में बदलने के लिए किया जाता है।
- Python में कुछ built-in functions होती हैं जो data conversion में मदद करती हैं।

(A) Common Conversion Functions

Function	Description	Example	Output
int(x)	x को integer में convert करता है	int(25.9)	25
		int('100')	100
float(x)	x को float में convert करता है	float(10)	10.0
		float('3.14')	3.14
complex(x)	x को complex में convert करता है	complex(5)	(5+0j)
		complex('2+3j')	(2+3j)
str(x)	x को string में convert करता है	str(50)	'50'
		str(3.14)	'3.14'
bool(x)	x को boolean में convert करता है	bool(0)	False
		bool(25)	True



(B) Type Conversion Examples

```
>>> int(3.99)      → 3
>>> int('45')     → 45
>>> float('7.25')  → 7.25
>>> str(100)       → '100'
>>> str(3.5)       → '3.5'
>>> complex('4+6j') → (4+6j)
>>> bool(0)        → False
>>> bool(100)      → True
```

Important Points :

- * int() decimal value को truncate करता है।
- * float(), int() string को valid number होने पर ही convert करते हैं, वरना ValueError देते हैं।
- * bool(0), bool(0.0), bool(''), bool([]), bool({}) हमेशा False return करते हैं।
- * बाकी सभी values True मानी जाती हैं।

Note : सही data type का चयन program की efficiency और memory usage के लिए बहुत महत्वपूर्ण होता है।

1. Simple Output

- Python में output स्क्रीन पर print() function द्वारा display किया जाता है।
- print() function text (string), variables, numbers या expressions को output के रूप में दिखाता है।
- यह function end में newline (\n) add करता है।

Syntax :

```
print(value1, value2, ..., sep = ' ', end = '\n')
```

Default Values

sep = ' ' (space)
end = '\n' (newline)

Examples :

Code	Explanation	Output
① print("Hello World")	String को print करता है।	Hello World
② print(10)	Integer number को print करता है।	10
③ print(3.14)	Float number को print करता है।	3.14
④ print("Python", 2024)	Multiple values (default sep = space)	Python 2024
⑤ print("A", "B", "C", sep = '-')	Separator '-' के साथ output देता है।	A-B-C

Important Points :

- * print() automatically next line पर चला जाता है।
- * end parameter के द्वारा newline को change किया जा सकता है।
ex. print("Hello", end = ' ') → cursor next line पर नहीं जाएगा।
- * sep parameter के द्वारा values के बीच separator change किया जा सकता है।

More Examples :

```
>>> print("Hello", end = ' ')
>>> print("World")
Output → Hello World

>>> print(10, 20, 30, sep = ',')
Output → 10, 20, 30
```

2. Simple Input

- Python में user से data प्राप्त करने के लिए input() function का उपयोग किया जाता है।
- input() function हमेशा string के रूप में data return करता है।

Syntax :

```
variable = input(prompt)
```

Note

prompt user को बताता है।
कि कौन-सा input देना है।

Examples :

Code	Explanation	User Input (मान लें)	Output (Stored in Variable)
① name = input("Enter your name: ")	User से name input लो	Pankaj	name = 'Pankaj'
② age = input("Enter your age: ")	User से age input लो	20	age = '20'
③ num = input("Enter a number: ")	User से number input लो	55	num = '55'

Type Conversion Example :

```
num = input("Enter a number: ")
print("Type of num:", type(num)) # String
num = int(num)
print("After conversion:", type(num)) # Integer
```

Important Points :

- * input() हमेशा string return करता है।
- * Numerical calculations के लिए type conversion करना आवश्यक होता है।
- * Prompt के बिना input() लेने पर cursor अगली line में चला जाता है।

Complete Example :

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Hello", name)
print("Next year your age will be", age + 1)
```

User Input (मान लें)

Pankaj
20

Output

Hello Pankaj
Next year your age will be 21

1. The % Method

- Python में % operator का उपयोग string formatting (old style formatting) के लिए किया जाता है।
- इसमें format string के अंदर % के साथ placeholders होते हैं और values को tuple के रूप में दिया जाता है।
- यह method Python 2.x से use में था और आज भी backward compatibility के लिए available है।

Syntax :

```
"format string" % (value1, value2, ...)
```

Placeholders :

Placeholder	Data Type	Example	Meaning
%d	Integer	"Age: %d" % 20	Integer number
%f	Float	"Price: %f" % 99.50	Float number
%.2f	Float (precision)	"PI: %.2f" % 3.14159	Float with 2 decimal places
%s	String	"Name: %s" % "Pankaj"	String
%c	Character	"Grade: %c" % 'A'	Single character
%%	Percent Sign	"Discount: 50%% off"	Prints % sign

Examples :

- ① "Hello %s" % "Pankaj"
- ② "Age = %d years" % 20
- ③ "PI value = %.2f" % 3.14159
- ④ "Name: %s, Age: %d" % ("Pankaj", 20)
- ⑤ "Total = %d, Price = %0.2f" % (2, 199.956)

Output :

- Hello Pankaj
- Age = 20 years
- PI value = 3.14
- Name: Pankaj, Age: 20
- Total = 2, Price = 199.96

2. The print() Function

- print() function Python में output display करने के लिए use होता है।
- यह function values, variables, expressions आदि को screen पर दिखाता है।
- print() function end में newline add करता है, जिससे अगला output अगली line में आता है।

Syntax :

```
print(value1, value2, ..., sep=' ', end='\n')
```

Parameters :

- value1, value2, ... : print करने के लिए values या variables
- sep : values के बीच separator (default space)
- end : line के अंत में add करने वाला character (default newline)

Default Parameters

sep = ' ' (space)
end = '\n' (newline)

Examples :

Code	Output
print("Hello")	Hello
print(10)	10
print("Pankaj", 20)	Pankaj 20
print("A", "B", "C")	A B C
print("Hello", end=' ') print("World")	Hello World (same line)
print(1, 2, 3, sep='-')	1-2-3

More Examples :

- * print("Hi", "Python", sep='***')
Output → Hi***Python
- * print("Line 1")
print("Line 2")
Output → Line 1
Line 2
- * print("End", end='!')
print("Done")
Output → End!Done

Note : * Python 3 में f-string (formatted string literal) का use ज्यादा होता है, लेकिन % method और print() function still useful हैं।
* print() function debugging और output display के लिए बहुत important है।